Teach Module 39: Spring Data JPA and Oracle Integration

This guide was created from the chat session for Module 39. The course document was used only to identify the module topic. The teaching content below is original practice-oriented instruction.

Module Topic

Spring Data JPA and Oracle Integration

Spring Data JPA lets a Java/Spring Boot application work with database tables using Java objects instead of writing SQL for every operation. Oracle is the relational database.

```
Java class  <->  JPA Entity  <->  Database table
Repository  <->  CRUD/query layer
Service     <->  Business logic
Controller  <->  API endpoint
```

1. What Problem Does JPA Solve?

Without JPA, Java database code often requires manual SQL, prepared statements, result sets, and object mapping.

With JPA, you model a table as a Java class:

```java
@Entity
@Table(name = "EMPLOYEES")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "FIRST_NAME")
    private String firstName;

    @Column(name = "EMAIL")
    private String email;
}
```

Spring Data JPA can then save, find, update, and delete records with much less repetitive code.

2. Entity Classes

An entity is a Java class mapped to a database table.

Important annotations:

```java
@Entity
```

Marks the class as a JPA-managed database entity.

```
@Table(name = "CUSTOMERS")
```

Maps the class to a specific table.

```
@Id
```

Marks the primary key.

```
@Column(name = "EMAIL")
```

Maps a Java field to a database column.

Example:

```
@Entity
@Table(name = "CUSTOMERS")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "FULL_NAME", nullable = false)
    private String fullName;

    @Column(name = "EMAIL", unique = true)
    private String email;
}
```

Conceptually:

```
Customer class       CUSTOMERS table
-------------        ----------------
id             ->    ID
fullName       ->    FULL_NAME
email          ->    EMAIL
```

3. Repository Interfaces

A repository is where database operations live.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
}
```

This gives you methods automatically:

```
findAll()
findById(id)
```

```
save(customer)
deleteById(id)
count()
existsById(id)
```

Example usage:

```
Customer customer = new Customer();
customer.setFullName("Asha Patel");
customer.setEmail("asha@example.com");

customerRepository.save(customer);
```

Spring Data JPA generates the implementation behind the scenes.

4. Query Derivation

Spring can create queries from method names.

```
List<Customer> findByEmail(String email);
```

Spring understands this as a query that finds customers where the email matches the provided value.

More examples:

```
List<Customer> findByFullNameContaining(String name);

List<Customer> findByEmailEndingWith(String domain);

List<Customer> findByFullNameIgnoreCase(String fullName);

boolean existsByEmail(String email);
```

This is called **derived query methods**.

5. Custom JPQL Queries

JPQL is similar to SQL, but it uses entity names and Java field names instead of table and column names.

```
@Query("SELECT c FROM Customer c WHERE c.email = :email")
Optional<Customer> findCustomerByEmail(@Param("email") String email);
```

Notice:

```
Customer = Java entity
c.email  = Java field
```

6. Native Oracle Queries

Sometimes you need real Oracle SQL.

```
@Query(
    value = "SELECT * FROM CUSTOMERS WHERE EMAIL = :email",
    nativeQuery = true
)
Optional<Customer> findByEmailNative(@Param("email") String email);
```

Use native queries when you need Oracle-specific features or highly optimized SQL. Prefer JPQL or derived queries when possible because they are usually easier to maintain.

7. Pagination and Sorting

If a table has 100,000 rows, you usually do not want to return all rows at once.

Spring Data uses `Pageable`.

```
Page<Customer> findByFullNameContaining(String name, Pageable pageable);
```

Service example:

```
Pageable pageable = PageRequest.of(0, 20, Sort.by("fullName").ascending());

Page<Customer> page = customerRepository.findByFullNameContaining("a", pageable);
```

This means:

```
Page 0
20 records
Sorted by fullName ascending
```

A `Page` gives useful metadata:

```
page.getContent();
page.getTotalElements();
page.getTotalPages();
page.hasNext();
```

8. Transactions

A transaction groups database operations into one unit of work.

```
@Transactional
public void registerCustomer(Customer customer) {
    customerRepository.save(customer);
    auditRepository.save(new AuditLog("Customer created"));
}
```

If saving the audit log fails, the customer save can be rolled back too.

Think of a transaction as:

```
Either all database changes succeed,
or none of them are committed.
```

In Spring, `@Transactional` is usually placed on service methods, not controllers.

9. Oracle Integration in Spring Boot

Typical Oracle configuration in `application.properties`:

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521/XEPDB1
spring.datasource.username=app_user
spring.datasource.password=secret
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.OracleDialect
```

Common `ddl-auto` values:

```
none      Spring does not touch schema
validate  Checks entity mappings against existing tables
update    Attempts to update schema automatically
create    Drops and recreates schema
```

For enterprise Oracle apps, `validate` or `none` is usually safer than `update` or `create`.

10. Common Mistakes

A common mistake is putting business logic inside repositories. Repositories should focus on database access. Services should contain business rules.

Good structure:

```
Controller: receives API request
Service: applies business rules
Repository: talks to database
Entity: maps to table
```

Another common mistake is exposing entities directly from REST APIs. For real applications, prefer DTOs.

Entity:

```
public class Customer {
    private Long id;
    private String fullName;
```

```
    private String email;
}
```

DTO:

```
public record CustomerResponse(
    Long id,
    String fullName,
    String email
) {}
```

Practice Exercises

Exercise 1: Create a Basic Entity

Create a `Customer` entity mapped to a `CUSTOMERS` table.

Fields:

```
id
firstName
lastName
email
phoneNumber
createdAt
```

Goal: understand `@Entity`, `@Table`, `@Id`, and `@Column`.

Exercise 2: Build a Repository

Create:

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
}
```

Then practice:

```
save()
findAll()
findById()
deleteById()
existsById()
```

Goal: use built-in Spring Data JPA CRUD methods.

Exercise 3: Derived Query Methods

Add custom repository methods:

```
List<Customer> findByLastName(String lastName);

Optional<Customer> findByEmail(String email);

List<Customer> findByFirstNameContainingIgnoreCase(String keyword);

boolean existsByEmail(String email);
```

Goal: learn how Spring creates queries from method names.

Exercise 4: Product Search with Sorting

Create a `Product` entity:

```
id
name
category
price
inStock
```

Repository methods:

```
List<Product> findByCategory(String category);

List<Product> findByInStockTrue();

List<Product> findByCategoryOrderByPriceAsc(String category);
```

Goal: practice filtering and ordering.

Exercise 5: Pagination

Create an endpoint that returns products page by page:

```
@GetMapping("/products")
public Page<Product> getProducts(
        @RequestParam int page,
        @RequestParam int size
) {
    Pageable pageable = PageRequest.of(page, size);
    return productRepository.findAll(pageable);
}
```

Goal: understand `Pageable`, `Page`, page number, page size, and total results.

Exercise 6: Custom JPQL Query

Add a method to find expensive products:

```java
@Query("SELECT p FROM Product p WHERE p.price > :minPrice")
List<Product> findProductsAbovePrice(@Param("minPrice") BigDecimal minPrice);
```

Goal: practice JPQL using entity names and field names.

Exercise 7: Native Oracle Query

Write a native SQL query:

```java
@Query(
    value = "SELECT * FROM PRODUCTS WHERE CATEGORY = :category",
    nativeQuery = true
)
List<Product> findByCategoryNative(@Param("category") String category);
```

Goal: understand when you are using real database SQL instead of JPQL.

Exercise 8: Transaction Practice

Create an `Order` and `OrderItem` flow.

Tables/entities:

```
Order
OrderItem
Product
```

Service method:

```java
@Transactional
public Order placeOrder(Long customerId, List<Long> productIds) {
    // create order
    // add order items
    // reduce product stock
    // save everything
}
```

Goal: learn why multi-step database operations need `@Transactional`.

Exercise 9: Validation and Error Handling

Add rules:

```
Email must be unique
Price must be greater than 0
Product name cannot be blank
Customer must exist before placing order
```

Goal: combine JPA with service-layer validation.

Exercise 10: Mini Project

Build a small **Inventory Management API**.

Entities:

```
Product
Supplier
PurchaseOrder
PurchaseOrderItem
```

Features:

```
Create product
Update product price
Find products by category
Find products below stock threshold
Paginate product list
Create purchase order
Use transaction for purchase order creation
```

Recommended endpoints:

```
POST /products
GET /products
GET /products/{id}
GET /products/search?category=Books
GET /products/low-stock
POST /purchase-orders
```

Lab: Customer Orders API

Build a small **Customer Orders API** using Spring Boot, Spring Data JPA, and Oracle.

Goal

Create an API that can:

```
Create customers
Create products
Place orders
View customer orders
Search products
Use pagination
Use transactions
```

Database Model

Use these entities:

```
Customer
Product
CustomerOrder
OrderItem
```

Customer

```
id
firstName
lastName
email
phoneNumber
```

Product

```
id
name
category
price
stockQuantity
```

CustomerOrder

```
id
customer
orderDate
status
totalAmount
```

OrderItem

```
id
order
product
quantity
unitPrice
lineTotal
```

Required Tasks

1. Create a Spring Boot project with these dependencies:

```
Spring Web
Spring Data JPA
Oracle JDBC Driver
```

```
Validation
Lombok optional
```

2. Configure Oracle in `application.properties`:

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521/XEPDB1
spring.datasource.username=app_user
spring.datasource.password=secret
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.OracleDialect
```

3. Create JPA entities for `Customer`, `Product`, `CustomerOrder`, and `OrderItem`.

4. Create repositories:

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    Optional<Customer> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByCategory(String category);
    Page<Product> findByCategory(String category, Pageable pageable);
    List<Product> findByStockQuantityLessThan(Integer quantity);
}
```

```
public interface CustomerOrderRepository extends JpaRepository<CustomerOrder, Long> {
    List<CustomerOrder> findByCustomerId(Long customerId);
}
```

5. Add a transactional order service:

```
@Transactional
public CustomerOrder placeOrder(Long customerId, List<OrderItemRequest> items) {
    // find customer
    // find products
    // check stock
    // calculate totals
    // reduce stock
    // save order
}
```

6. Create REST endpoints:

```
POST /customers
GET /customers/{id}
GET /customers/by-email?email=

POST /products
GET /products
GET /products/search?category=
GET /products/low-stock?threshold=

POST /orders
GET /orders/customer/{customerId}
```

Sample Order Request

```
{
  "customerId": 1,
  "items": [
    {
      "productId": 10,
      "quantity": 2
    },
    {
      "productId": 12,
      "quantity": 1
    }
  ]
}
```

Business Rules

```
Customer email must be unique.
Product price must be greater than 0.
Product stock quantity cannot be negative.
Order cannot be placed if product stock is insufficient.
Order total must equal the sum of all line totals.
Order creation must be transactional.
```

Stretch Tasks

Add:

```
JPQL query for products above a given price
Native Oracle query for products by category
Pagination and sorting for product list
DTOs instead of returning entities directly
Global exception handling with @ControllerAdvice
```

Expected Outcome

By the end of this lab, you should understand how Spring Data JPA maps Java classes to Oracle tables, how repositories reduce database code, how transactions protect order creation, and how pagination/custom queries work in a real API.